- 딥러닝
- = 손실 함수를 최소화 하는 거임

- **Gradient Descent**가 쓰임
- 최적인 세타를 찾아야 함.
- 기울기의 반대 방향으로 이동하며 → 손실 함수의 최소값을 찾음.
- $\theta \leftarrow \theta - \eta \nabla L(\theta)$
- $\nabla L(\theta)$ → 손실 함수가 가장 빠르게 증가하는 방향
- $-\nabla L(\theta)$ → 손실 함수가 가장 빠르게 감소하는 방향
- η → learning rate (한 번에 이동하는 크기 조절)
- 전역 최소값에 도달하는 것은 보장되지 않음.

$$\theta^{k+1} \leftarrow \theta^k - \eta \nabla_\theta J(\theta^k)$$

- → one iteration.

Stochastic Gradient Descent(SGD)

- 데이터가 너무 많아서, 학습하기 힘들 때가 많음
- 전체 데이터 대신 일부 데이터를 무작위로 뽑아서 gradient를 계산함.
- 엄밀한 SGD → 샘플 1개로 gradient 계산
- Mini-batch GD → 여러 샘플로 이루어진 작은 batch로 gradient 계산
- 전체 데이터 N개에 대한 평균 손실이 $L(\theta) = (1/N) \Sigma_{i=1}^{N} \ell_i(\theta)$ 이면,
- 전체 gradient: $g = (1/N) \Sigma_{i=1}^{N} \nabla \ell_i(\theta)$
- Mini-batch gradient: $g_B = (1/|B|) \Sigma_{i \in B} \nabla \ell_i(\theta)$
- 무작위로 균등 추출하면 → $E[g_B] = g$
- 즉, mini-batch gradient는 전체 gradient의 불편추정량(unbiased estimator)임.
- 매번 똑같거나 비슷하다고 가정하는 게 아니라, 기댓값이 같다는 뜻임.
- 일부 데이터만 계산하므로 한 번의 업데이트 비용이 작음.
- 하지만 sampling에 따른 분산이 있어서 → 업데이트 방향이 흔들리고 학습이 불안정할 수 있음.

If batchsize=1 then θ is updated after each example.

If batchsize=N (full set) then this is standard gradient descent.

- 이렇게 하면 빠름. + 로컬 minimum에서 벗어날 수 있음
  좀 불안정함.

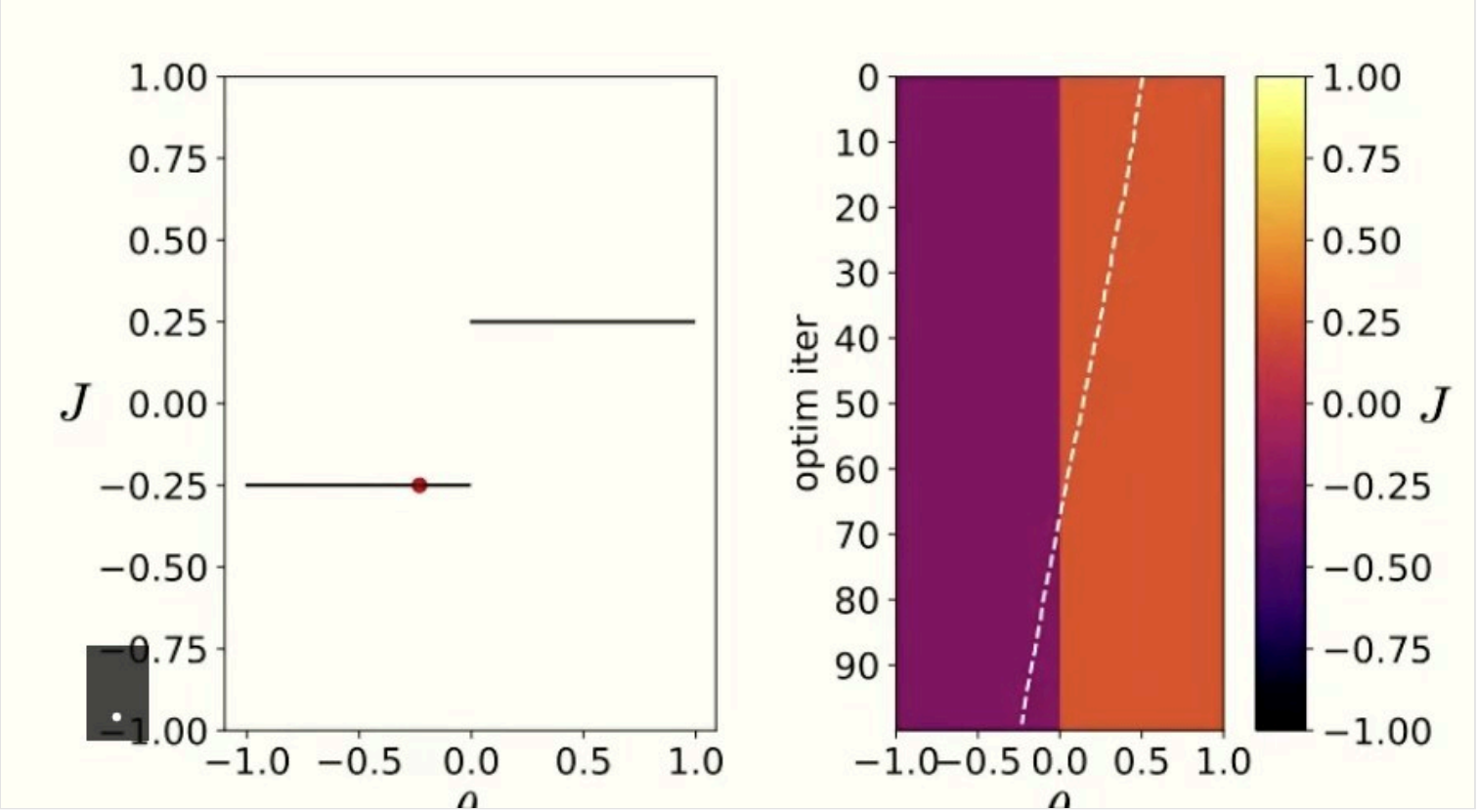
Momentum

- GD에서 업데이트 방향으로 **관성**을 가지고 업데이트 하게 됨

$$\theta^{t+1} \leftarrow \theta^t - \eta \nabla f(\theta^t) - \alpha m^t$$

- 너무 작은 모멘텀 → 최소값에 적당히 천천히 감
  적당한 모멘텀 → 최소값에 더 빨리 도달 할 수 있음
  너무 높은 모멘텀 → 최소값을 지나가버림

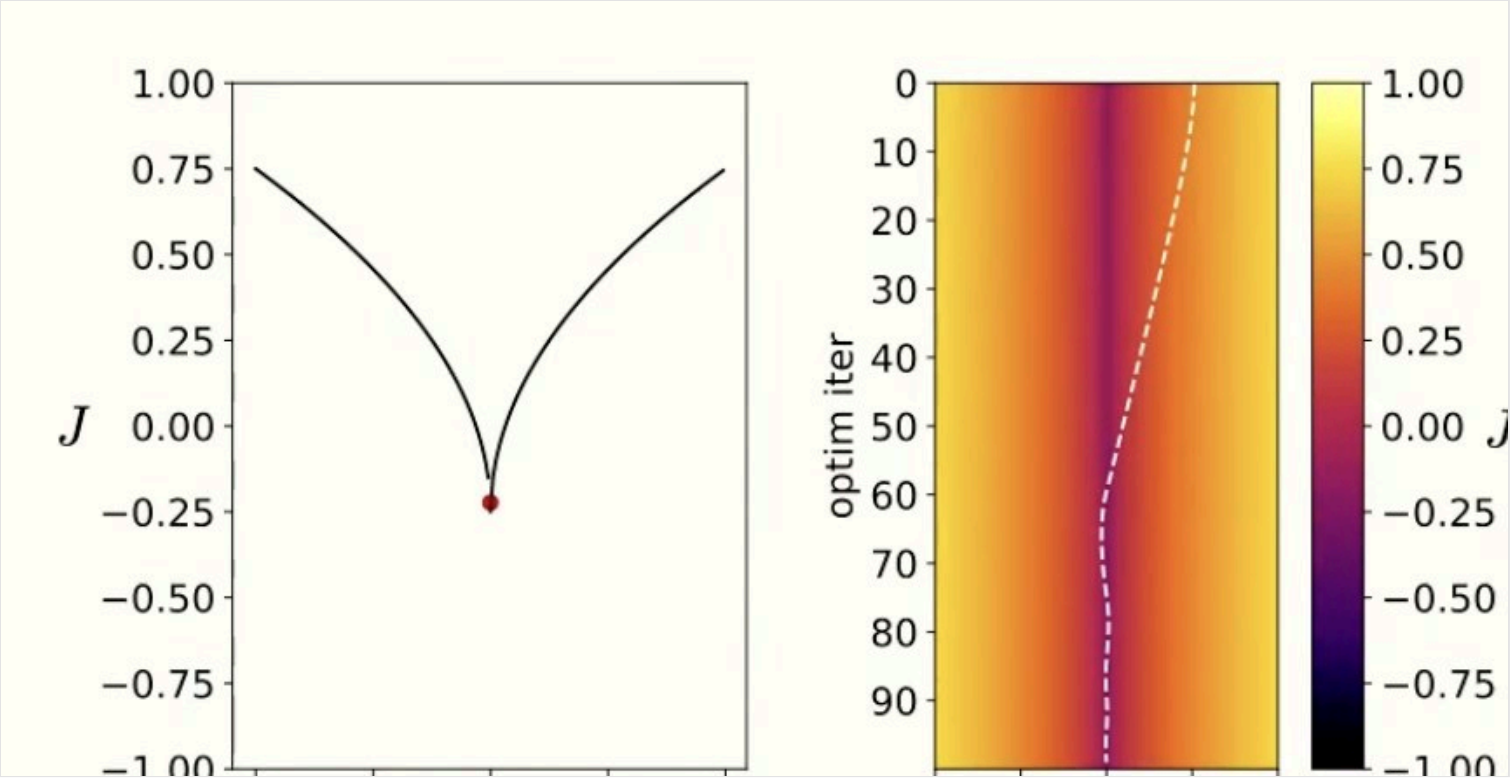
Evolution Strategies



$$\epsilon_i \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$$
$$s_i = J(\theta + \sigma \epsilon_i)$$

$$\theta^{k+1} \leftarrow \theta^k - \eta \frac{1}{\sigma M} \sum_{i=1}^{M} s_i \epsilon_i$$

Gradient clipping



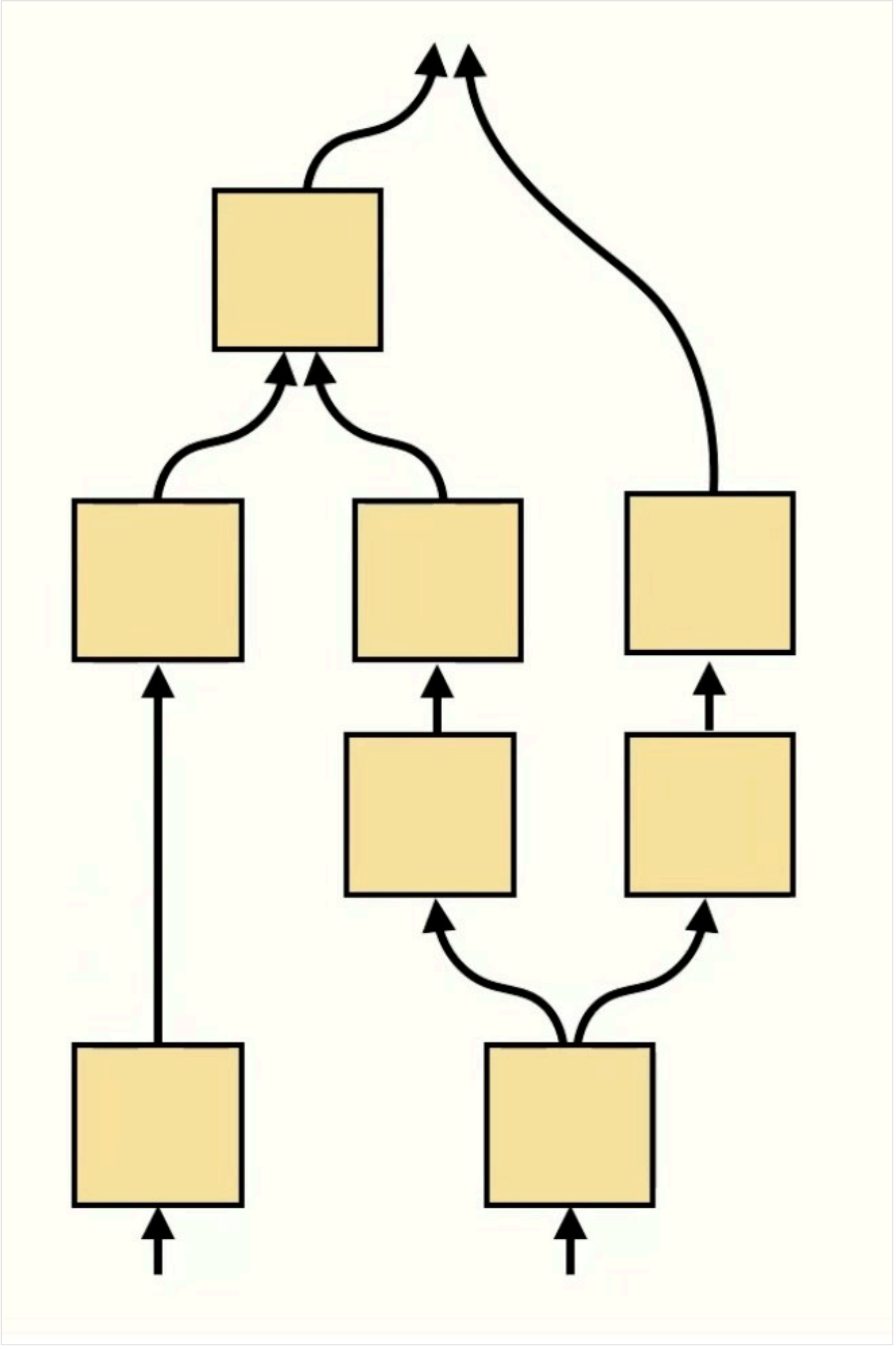
- 기울기가 너무 커지면 → 기울기 자체를 제한함.
- 매개변수 변화량이 아니라 gradient가 1차적인 clipping 대상임.
- $g = \nabla L(\theta)$, $c > 0$ (clipping threshold)
- **Norm clipping**
- $\|g\| > c$이면 전체 gradient를 재조정함.
- $g_{clip} = g \cdot \min(1, c/\|g\|)$
- → gradient의 norm을 최대 c로 제한함.
- → 원래 gradient의 방향은 유지됨.
- **Value clipping**
- 각 gradient 성분을 [-c, c] 범위로 제한함.
- $(g_{clip})_i = \max(-c, \min(c, g_i))$
- → 각 성분을 개별적으로 잘라냄.
- → 전체 gradient의 방향은 바뀔 수 있음.
- Clipping한 gradient로 매개변수를 업데이트함.
- $\theta \leftarrow \theta - \eta g_{clip}$
- 즉, 너무 큰 gradient 때문에 업데이트가 과도해지는 것을 방지하는 방법임.

$$\mathbf{v} = \nabla_\theta J(\theta^k)$$
$$\theta^{k+1} \leftarrow \theta^k - \eta[\mathtt{clip}(v_1, -m, m), \ldots, \mathtt{clip}(v_M, -m, m)]^\mathsf{T}$$

Loss function

- 손실 함수를 최소화하기 위해 gradient를 계산함.
- 전 구간에서 미분 가능하거나 smooth할 필요는 없음.
- 대부분의 점에서 미분 가능하면 gradient 기반 학습이 가능하고, 미분 불가능한 점에서도 부분기울기(subgradient)가 정의되면 이를 이용할 수 있음.
- EX> ReLU = max(0, z)
- ReLU는 손실 함수가 아니라 활성화 함수(activation function)임.
- z = 0에서 미분 불가능하지만, 해당 지점에서 사용할 기울기 값을 정해 학습할 수 있음.
- GELU는 smooth한 활성화 함수임.
- ReLU 대신 GELU를 쓰기도 하지만, 선택 이유가 smooth함 하나만은 아님.

Computation Graphs



- 아래가 input, 위로 갈수록 output

Backpropagation

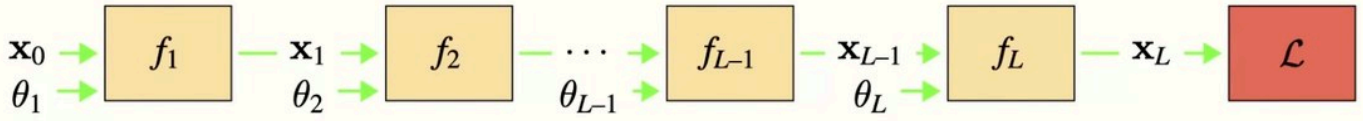
$$\frac{\partial J}{\partial \theta_1} = \frac{\partial J}{\partial \mathbf{x}_L}\frac{\partial \mathbf{x}_L}{\partial \mathbf{x}_{L-1}}\cdots\frac{\partial \mathbf{x}_3}{\partial \mathbf{x}_2}\frac{\partial \mathbf{x}_2}{\mathbf{x}_1}\frac{\partial \mathbf{x}_1}{\partial \theta_1}$$

$$\frac{\partial J}{\partial \theta_2} = \frac{\partial J}{\partial \mathbf{x}_L}\frac{\partial \mathbf{x}_L}{\partial \mathbf{x}_{L-1}}\cdots\frac{\partial \mathbf{x}_3}{\partial \mathbf{x}_2}\frac{\partial \mathbf{x}_2}{\partial \theta_2}$$

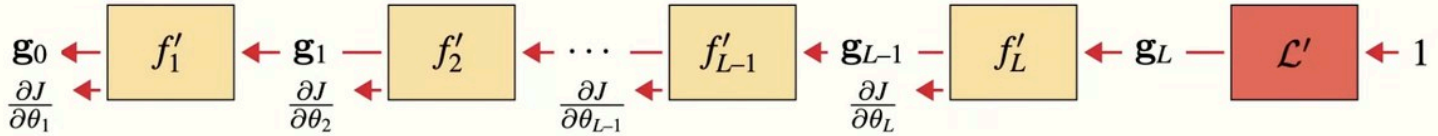
- 정정: 슬라이드의 연쇄법칙 식에서 끝에서 두 번째 인자인 ∂x₂/x₁은 분모의 ∂가 누락된 오기입니다. 올바른 표기는 ∂x₂/∂x₁입니다.
- Computation Graphs에서 세타에 대한 미분을 계산하기 위한 방법.
  Chain rule을 이용해 backward 방향으로 미분값을 얻음

The Full Algorithm: Forward, Then Backward

Forward:



Backward:



Update:

$$\theta^{i+1} \leftarrow \theta^i - \eta\left(\frac{\partial J}{\partial \theta}\right)^{\mathsf{T}}$$

… and repeat

- forward → backward → update 반복하면 training

- Linear layer
- Forward → f = Wx + b
- W → weight matrix
- x → input vector
- b → bias vector
- f → output vector
- 입력에 W를 곱하고 bias b를 더함.
- bias까지 포함하면 수학적으로는 affine transformation임.
- b = 0이면 f = Wx가 됨.
- Backprop

$$\mathbf{g}_{\text{in}} = \mathbf{g}_{\text{out}} \cdot \frac{\partial f(\mathbf{x}_{\text{in}}, \mathbf{W})}{\partial \mathbf{x}_{\text{in}}} = \mathbf{g}_{\text{out}} \cdot \frac{\partial \mathbf{x}_{\text{out}}}{\partial \mathbf{x}_{\text{in}}}$$

$$\frac{\partial \mathbf{x}_{\text{out}_i}}{\partial \mathbf{x}_{\text{in}_j}} = \mathbf{W}_{ij} \longrightarrow \frac{\partial f(\mathbf{x}_{\text{in}}, \mathbf{W})}{\partial \mathbf{x}_{\text{in}}} = \mathbf{W}$$

- therefore

$$\mathbf{g}_{\text{in}} = \mathbf{g}_{\text{out}} \cdot \mathbf{W}$$

- loss function을 구하려면

$$\frac{\partial J}{\partial \mathbf{W}} = \mathbf{g}_{\text{out}} \cdot \frac{\partial f(\mathbf{x}_{\text{in}}, \mathbf{W})}{\partial \mathbf{W}} = \mathbf{g}_{\text{out}} \cdot \frac{\partial \mathbf{x}_{\text{out}}}{\partial \mathbf{W}}$$

- gout → chainrule 역할

$$\frac{\partial J}{\partial \mathbf{W}_{ij}} = \frac{\partial J}{\partial \mathbf{x}\mathbf{out}_i} \cdot \frac{\partial \mathbf{x}\mathbf{out}_i}{\partial \mathbf{W}_{ij}} = \frac{\partial J}{\partial \mathbf{x}\mathbf{out}_i} \cdot \mathbf{x}\mathbf{in}_j$$

$$\frac{\partial \mathbf{x}\mathbf{out}_i}{\partial \mathbf{W}_{ij}} = \mathbf{x}\mathbf{in}_j$$

$$\frac{\partial J}{\partial \mathbf{W}} = \mathbf{x}\mathbf{in} \cdot \frac{\partial J}{\partial \mathbf{x}\mathbf{out}} = \mathbf{x}\mathbf{in} \cdot \mathbf{g}\mathbf{out}$$

- ※ 정정: 캡처 15의 성분식 $\partial J/\partial W_{ij} = (g_{out})_i (x_{in})_j$가 기준임. 행렬로는 $\partial J/\partial W = g_{out}\, x_{in}^{T}$ (외적)이 맞음. 캡처 16의 $x_{in} \cdot g_{out}$는 일반적인 행렬곱 순서로 해석하면 안 됨.
- update 할 때 위 식을 쓰게 됨.

Differentiable programming

- Easy to optimize
  compositional “block based programming”